САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Мобильные и сетевые технологии

О Т Ч Е Т

Лабораторная работа №4

Тема работы: «Запросы на выборку и модификацию данных. Представления. Работа с индексами»

Обучающийся: Федорова Мария Витальевна, К3239
Поток: ПиРБД К24 МСТ 1.2
Преподаватель: Говорова М.М.

Санкт-Петербург,
2026

1. Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2. Практическое задание

В ходе выполнения лабораторной работы требовалось:

1. создать запросы и представления на выборку данных к базе данных PostgreSQL согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3;
2. составить 3 запроса на модификацию данных (`INSERT`, `UPDATE`, `DELETE`) с использованием подзапросов;
3. изучить графическое представление запросов и просмотреть историю запросов;
4. создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами.

3. Схема базы данных

В лабораторной работе №4 использовалась база данных `restaurant_service`, разработанная и реализованная в лабораторной работе №3. Индивидуальный вариант: **Вариант 13. БД «Ресторан»**.

В качестве схемы базы данных использовалась логическая модель, полученная с помощью инструмента Generate ERD в pgAdmin.

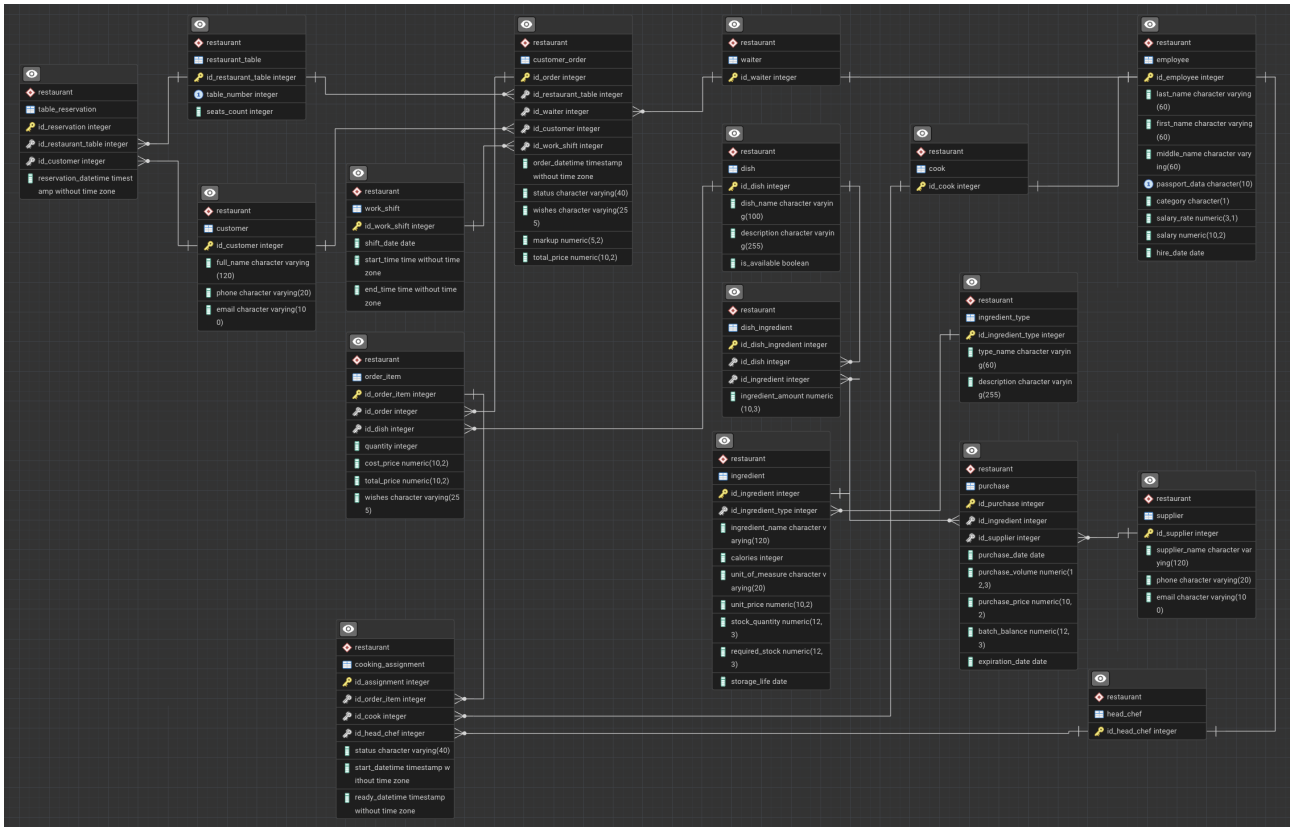


Рис. 1. ERD-схема базы данных, использованная в лабораторной работе №4

4. Выполнение

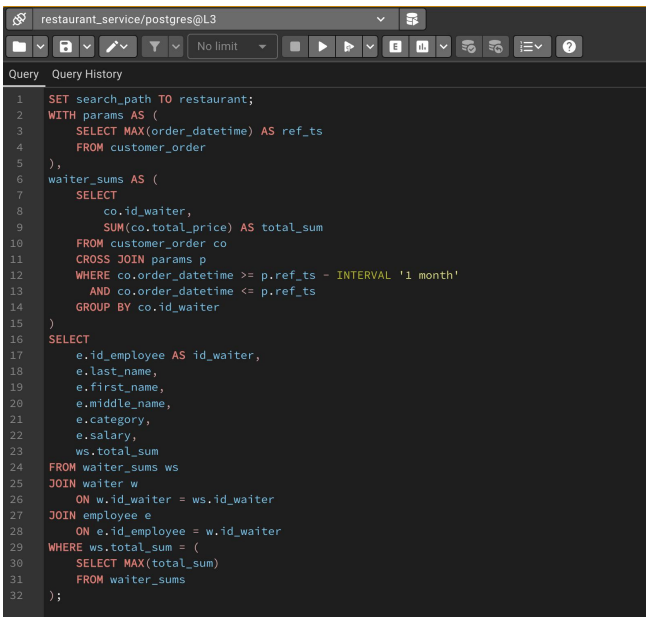
4.1. Запросы к базе данных

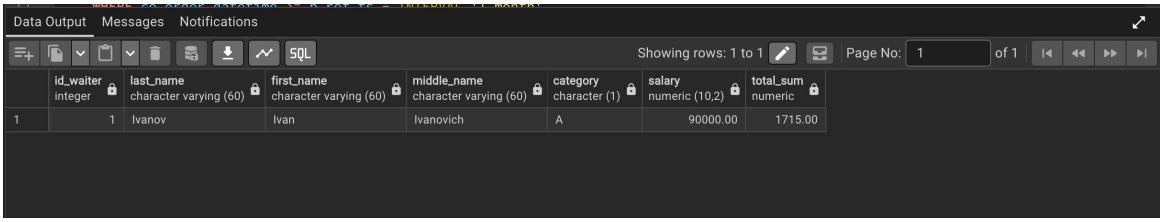
При выполнении запросов использовались данные, загруженные в лабораторной работе №3. Поскольку тестовые заказы относятся к дате 2026-03-20, для запросов с периодами использовалась максимальная дата заказа в таблице customer_order.

Запрос 1. Вывести данные официанта, принявшего заказы на максимальную сумму за истекший месяц

Листинг 1. Запрос на поиск официанта с максимальной суммой заказов за истекший месяц

```
SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime) AS ref_ts
    FROM customer_order
),
waiter_sums AS (
    SELECT
        co.id_waiter,
        SUM(co.total_price) AS total_sum
    FROM customer_order co
    CROSS JOIN params p
    WHERE co.order_datetime >= p.ref_ts - INTERVAL '1 month'
      AND co.order_datetime <= p.ref_ts
    GROUP BY co.id_waiter
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    e.middle_name,
    e.category,
    e.salary,
    ws.total_sum
FROM waiter_sums ws
JOIN waiter w
    ON w.id_waiter = ws.id_waiter
JOIN employee e
    ON e.id_employee = w.id_waiter
WHERE ws.total_sum = (
    SELECT MAX(total_sum)
    FROM waiter_sums
);
```





Запрос 2. Рассчитать премию каждого официанта за последние 10 дней

Листинг 2. Запрос на расчет премии каждого официанта за последние 10 дней

```
SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime)::date AS ref_date
    FROM customer_order
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    COALESCE(ROUND(SUM(co.total_price) * 0.05, 2), 0) AS bonus
FROM waiter w
JOIN employee e
    ON e.id_employee = w.id_waiter
LEFT JOIN customer_order co
    ON co.id_waiter = w.id_waiter
   AND co.order_datetime::date >= (
       SELECT ref_date - INTERVAL '9 days' FROM params
   )
   AND co.order_datetime::date <= (
       SELECT ref_date FROM params
   )
GROUP BY e.id_employee, e.last_name, e.first_name
ORDER BY e.id_employee;
```

restaurant_service/postgres@L3

No limit

Query | Query History

```sql
SET search_path TO restaurant;
WITH params AS (
    SELECT MAX(order_datetime)::date AS ref_date
    FROM customer_order
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    COALESCE(ROUND(SUM(co.total_price) * 0.05, 2), 0) AS bonus
FROM waiter w
JOIN employee e
    ON e.id_employee = w.id_waiter
LEFT JOIN customer_order co
    ON co.id_waiter = w.id_waiter
   AND co.order_datetime::date >= (
       SELECT ref_date - INTERVAL '9 days' FROM params
   )
   AND co.order_datetime::date <= (
       SELECT ref_date FROM params
   )
GROUP BY e.id_employee, e.last_name, e.first_name
ORDER BY e.id_employee
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

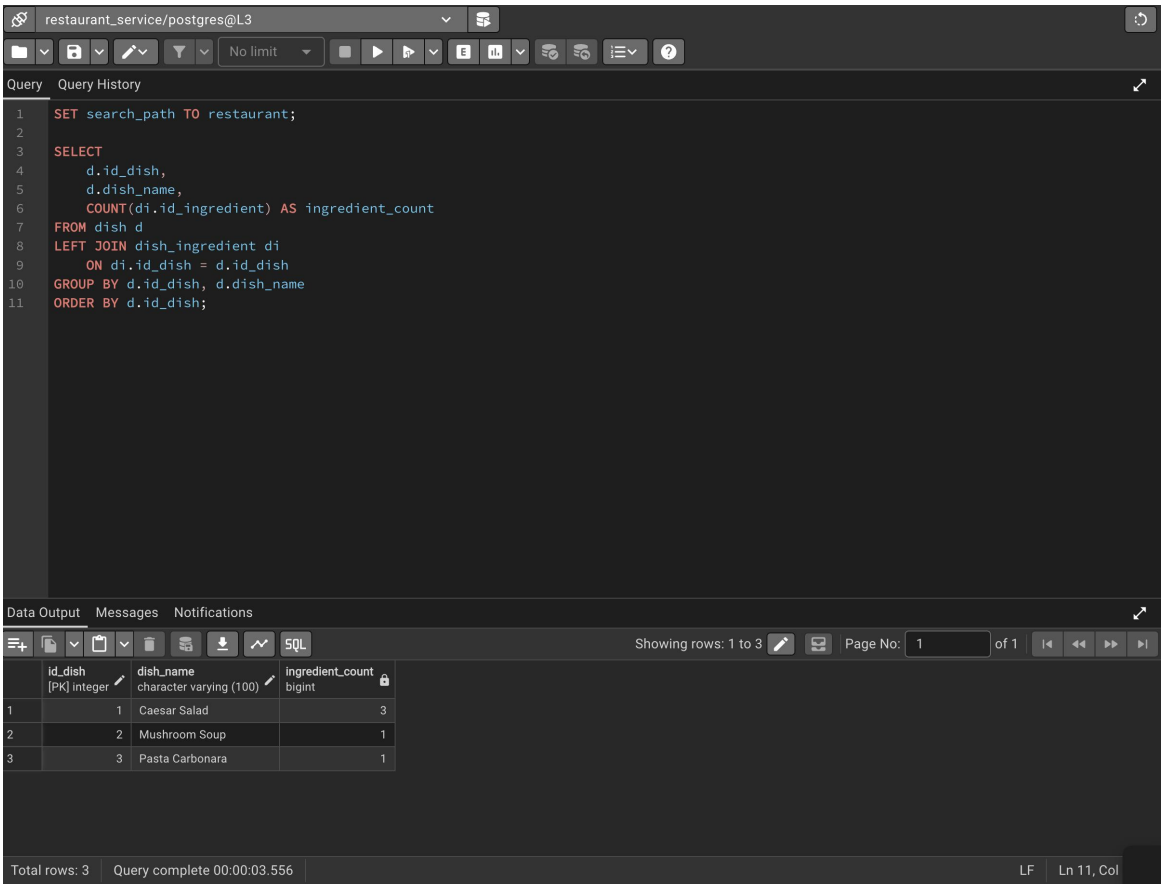
	id_waiter integer	last_name character varying (60)	first_name character varying (60)	bonus numeric
1	1	Ivanov	Ivan	85.75

Total rows: 1 | Query complete 00:00:02.874 | LF | Ln 1, Col 31

Запрос 3. Подсчитать, сколько ингредиентов содержит каждое блюдо

Листинг 3. Запрос на подсчет количества ингредиентов в каждом блюде

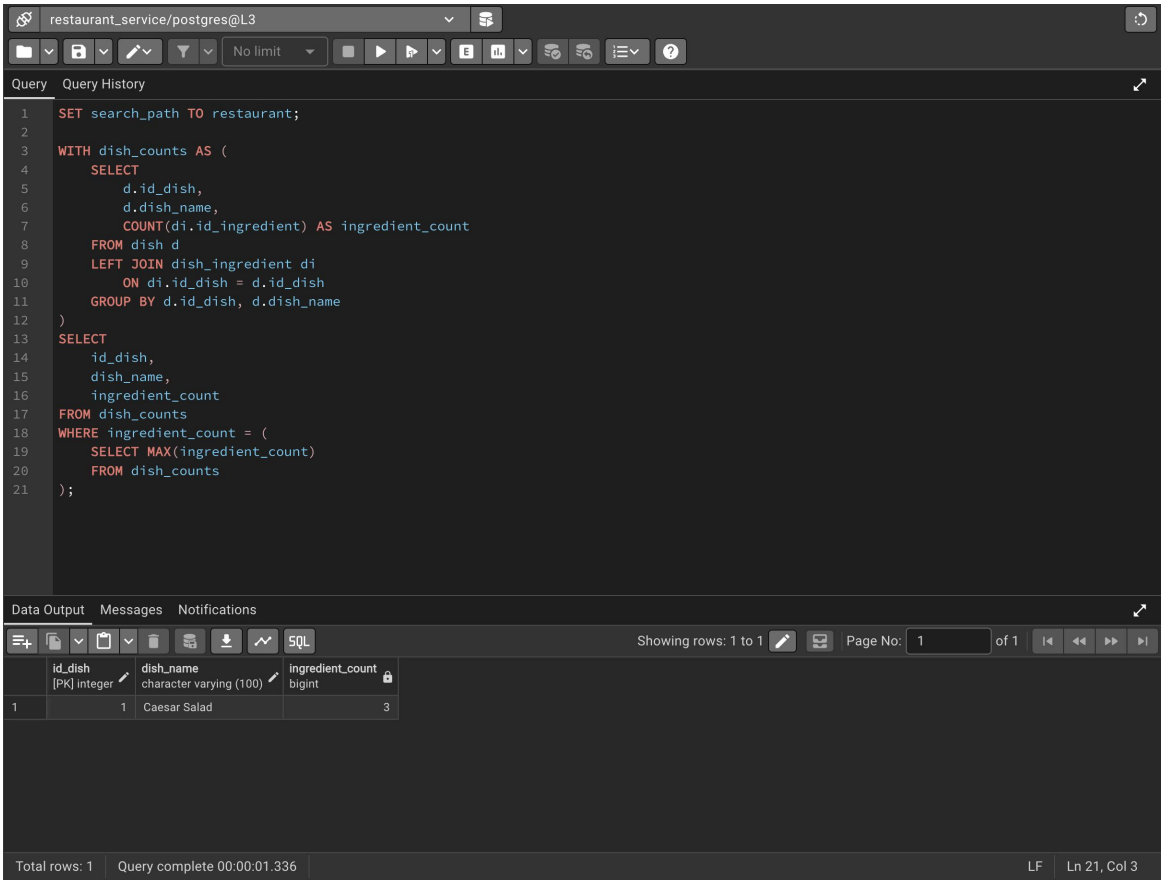
```
SET search_path TO restaurant;

SELECT
    d.id_dish,
    d.dish_name,
    COUNT(di.id_ingredient) AS ingredient_count
FROM dish d
LEFT JOIN dish_ingredient di
    ON di.id_dish = d.id_dish
GROUP BY d.id_dish, d.dish_name
ORDER BY d.id_dish;
```



Запрос 4. Вывести название блюда, содержащее максимальное число ингредиентов

Листинг 4. Запрос на поиск блюда с максимальным числом ингредиентов

```
SET search_path TO restaurant;

WITH dish_counts AS (
    SELECT
        d.id_dish,
        d.dish_name,
        COUNT(di.id_ingredient) AS ingredient_count
    FROM dish d
    LEFT JOIN dish_ingredient di
        ON di.id_dish = d.id_dish
    GROUP BY d.id_dish, d.dish_name
)
SELECT
    id_dish,
    dish_name,
    ingredient_count
FROM dish_counts
WHERE ingredient_count = (
    SELECT MAX(ingredient_count)
    FROM dish_counts
);
```

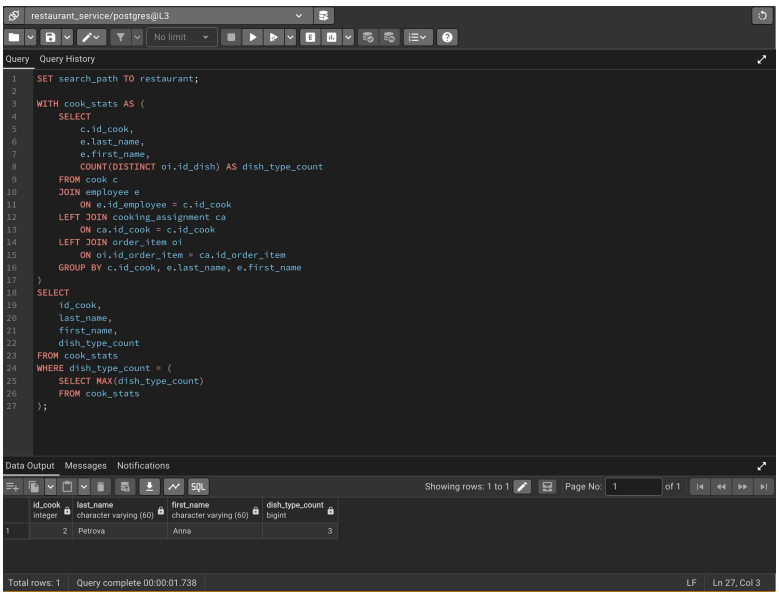


Запрос 5. Какой повар может приготовить максимальное число видов блюд

В реализованной структуре базы данных данный запрос интерпретировался как поиск повара, который фактически готовил максимальное число различных блюд, на основании таблиц `cooking_assignment`, `order_item` и `dish`.

Листинг 5. Запрос на поиск повара, приготовившего максимальное число видов блюд

```
SET search_path TO restaurant;

WITH cook_stats AS (
    SELECT
        c.id_cook,
        e.last_name,
        e.first_name,
        COUNT(DISTINCT oi.id_dish) AS dish_type_count
    FROM cook c
    JOIN employee e
        ON e.id_employee = c.id_cook
    LEFT JOIN cooking_assignment ca
        ON ca.id_cook = c.id_cook
    LEFT JOIN order_item oi
        ON oi.id_order_item = ca.id_order_item
    GROUP BY c.id_cook, e.last_name, e.first_name
)
SELECT
    id_cook,
    last_name,
    first_name,
    dish_type_count
FROM cook_stats
WHERE dish_type_count = (
    SELECT MAX(dish_type_count)
    FROM cook_stats
);
```

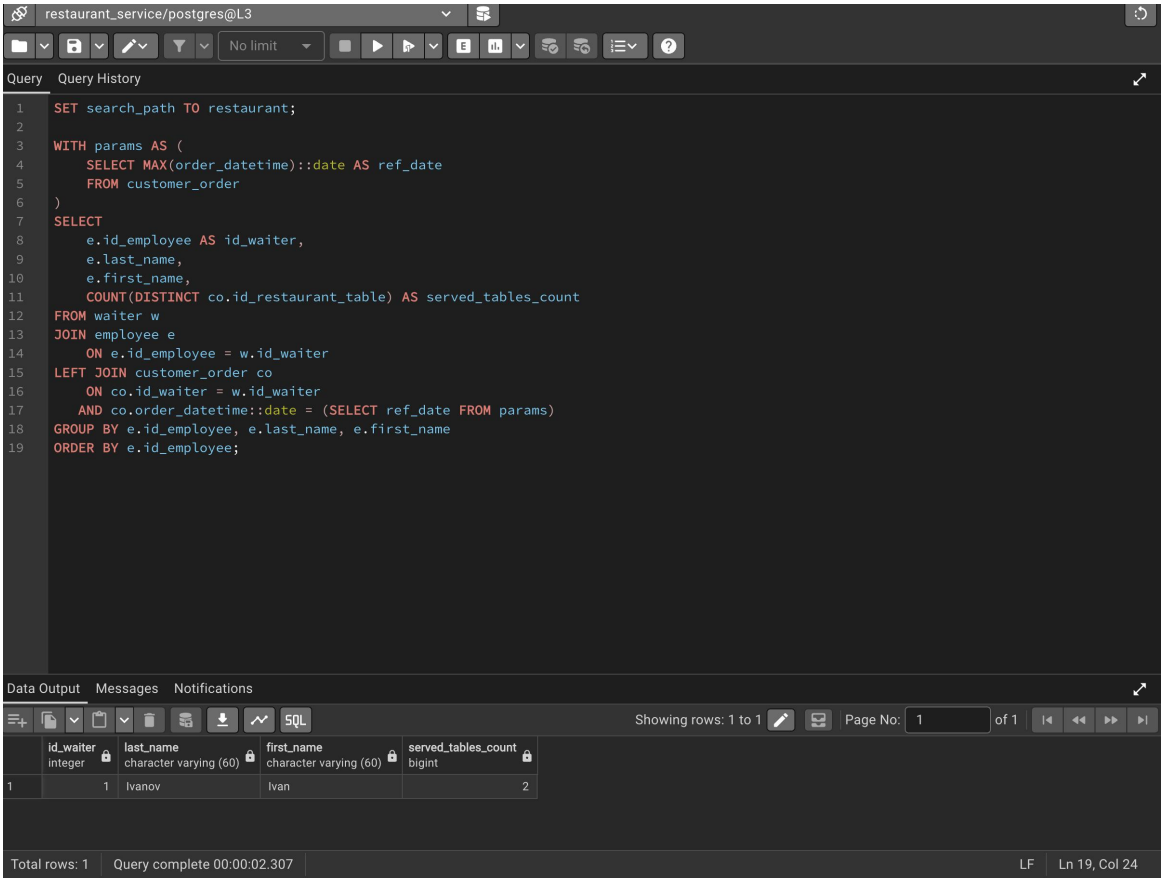


Запрос 6. Сколько закреплено столов за каждым из официантов за сегодняшний день

В текущей структуре базы данных отдельная таблица закрепления столов за официантами отсутствует, поэтому количество столов определялось как число уникальных столов, обслуженных официантом по заказам за дату, соответствующую максимальной дате заказа в таблице `customer_order`.

Листинг 6. Запрос на определение количества столов, обслуженных каждым официантом за день

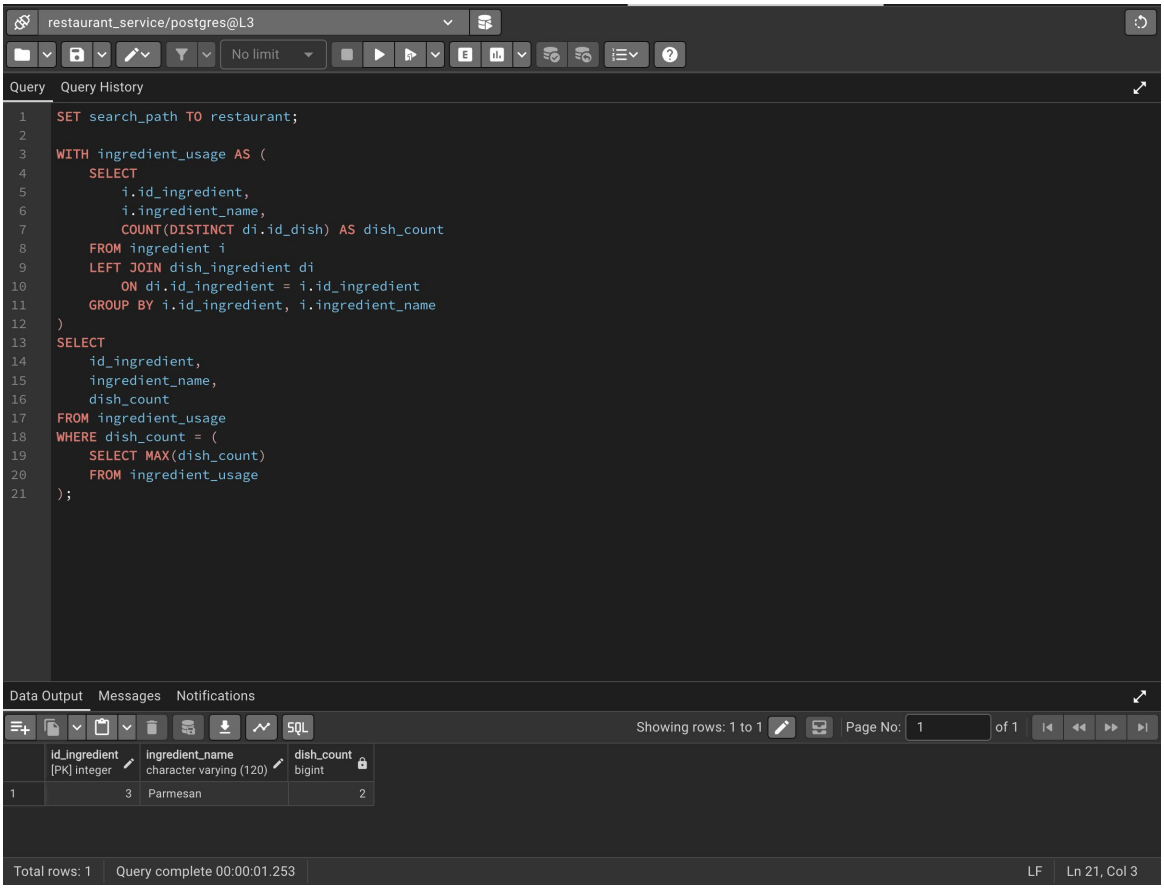
```
SET search_path TO restaurant;

WITH params AS (
    SELECT MAX(order_datetime)::date AS ref_date
    FROM customer_order
)
SELECT
    e.id_employee AS id_waiter,
    e.last_name,
    e.first_name,
    COUNT(DISTINCT co.id_restaurant_table) AS served_tables_count
FROM waiter w
JOIN employee e
    ON e.id_employee = w.id_waiter
LEFT JOIN customer_order co
    ON co.id_waiter = w.id_waiter
   AND co.order_datetime::date = (SELECT ref_date FROM params)
GROUP BY e.id_employee, e.last_name, e.first_name
ORDER BY e.id_employee;
```



Запрос 7. Какой из ингредиентов используется в максимальном количестве блюд

Листинг 7. Запрос на поиск ингредиента, используемого в максимальном количестве блюд

```
SET search_path TO restaurant;

WITH ingredient_usage AS (
    SELECT
        i.id_ingredient,
        i.ingredient_name,
        COUNT(DISTINCT di.id_dish) AS dish_count
    FROM ingredient i
    LEFT JOIN dish_ingredient di
        ON di.id_ingredient = i.id_ingredient
    GROUP BY i.id_ingredient, i.ingredient_name
)
SELECT
    id_ingredient,
    ingredient_name,
    dish_count
FROM ingredient_usage
WHERE dish_count = (
    SELECT MAX(dish_count)
    FROM ingredient_usage
);
```

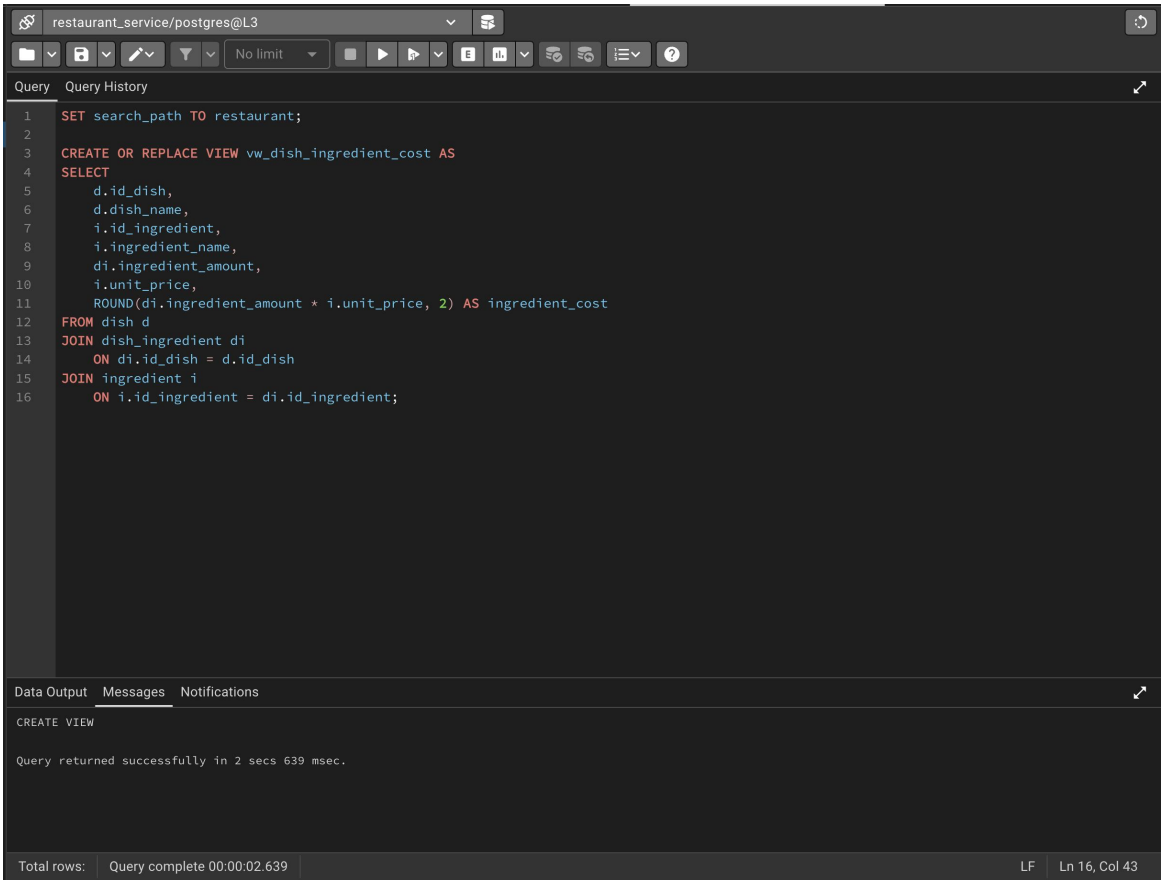


4.2. Представления

Представление 1. Для расчета стоимости ингредиентов для заданного блюда

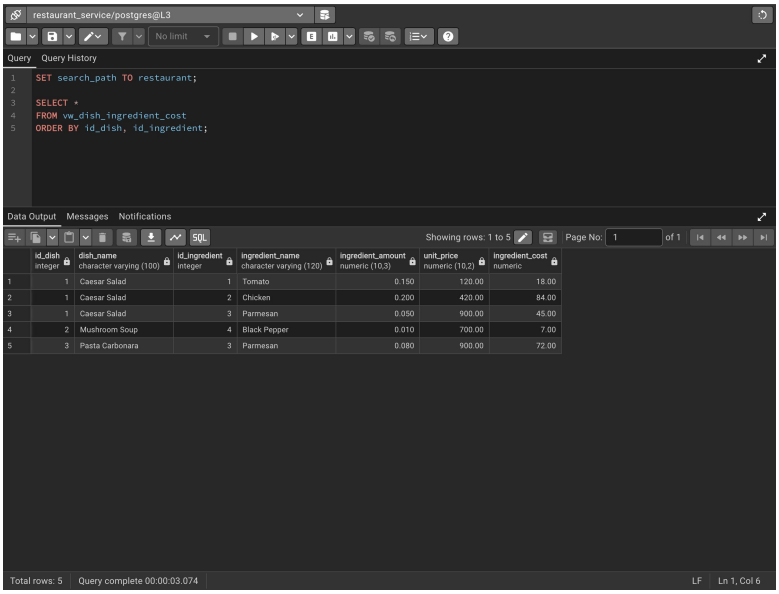
Листинг 8. Создание представления для расчета стоимости ингредиентов блюда

```
SET search_path TO restaurant;

CREATE OR REPLACE VIEW vw_dish_ingredient_cost AS
SELECT
    d.id_dish,
    d.dish_name,
    i.id_ingredient,
    i.ingredient_name,
    di.ingredient_amount,
    i.unit_price,
    ROUND(di.ingredient_amount * i.unit_price, 2) AS ingredient_cost
FROM dish d
JOIN dish_ingredient di
    ON di.id_dish = d.id_dish
JOIN ingredient i
    ON i.id_ingredient = di.id_ingredient;
```



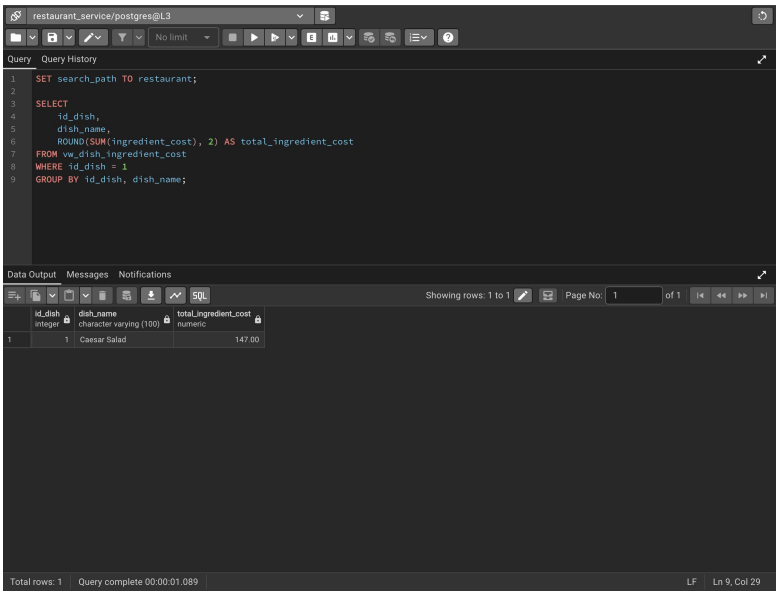
Листинг 9. Просмотр содержимого представления vw_dish_ingredient_cost

```
SET search_path TO restaurant;

SELECT *
FROM vw_dish_ingredient_cost
ORDER BY id_dish, id_ingredient;
```



Листинг 10. Расчет стоимости ингредиентов для блюда с id_dish = 1

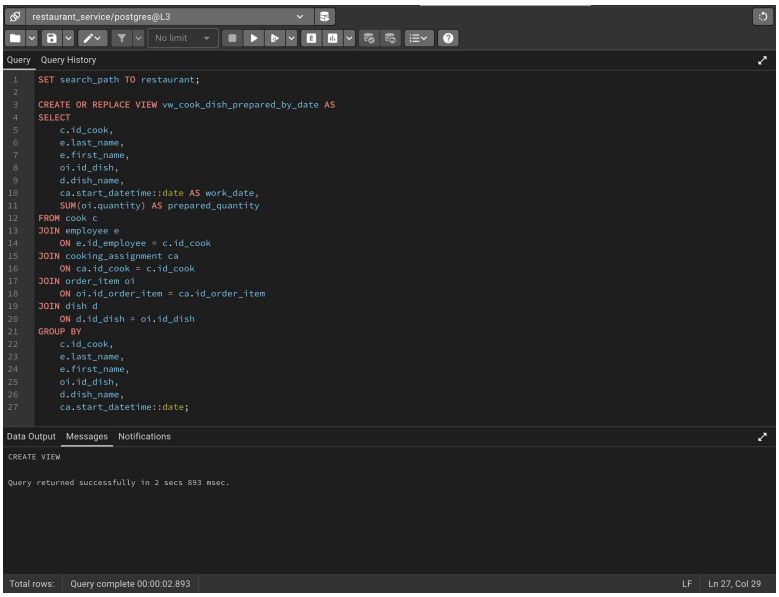
```
SET search_path TO restaurant;

SELECT
    id_dish,
    dish_name,
    ROUND(SUM(ingredient_cost), 2) AS total_ingredient_cost
FROM vw_dish_ingredient_cost
WHERE id_dish = 1
GROUP BY id_dish, dish_name;
```



Представление 2. Для всех поваров количество приготовленных блюд по каждому блюду за определенную дату

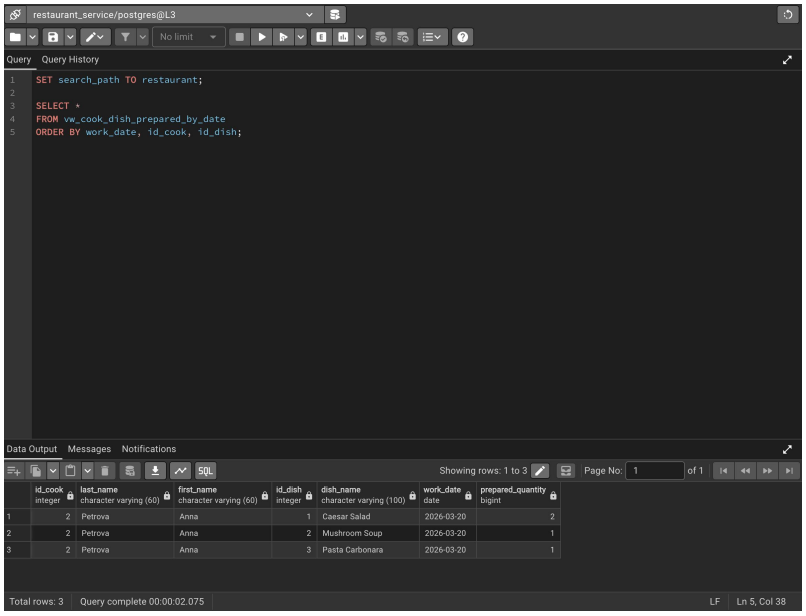
Листинг 11. Создание представления для определения количества приготовленных блюд по каждому повару и блюду

```
SET search_path TO restaurant;

CREATE OR REPLACE VIEW vw_cook_dish_prepared_by_date AS
SELECT
    c.id_cook,
    e.last_name,
    e.first_name,
    oi.id_dish,
    d.dish_name,
    ca.start_datetime::date AS work_date,
    SUM(oi.quantity) AS prepared_quantity
FROM cook c
JOIN employee e
    ON e.id_employee = c.id_cook
JOIN cooking_assignment ca
    ON ca.id_cook = c.id_cook
JOIN order_item oi
    ON oi.id_order_item = ca.id_order_item
JOIN dish d
    ON d.id_dish = oi.id_dish
GROUP BY
    c.id_cook,
    e.last_name,
    e.first_name,
    oi.id_dish,
    d.dish_name,
    ca.start_datetime::date;
```



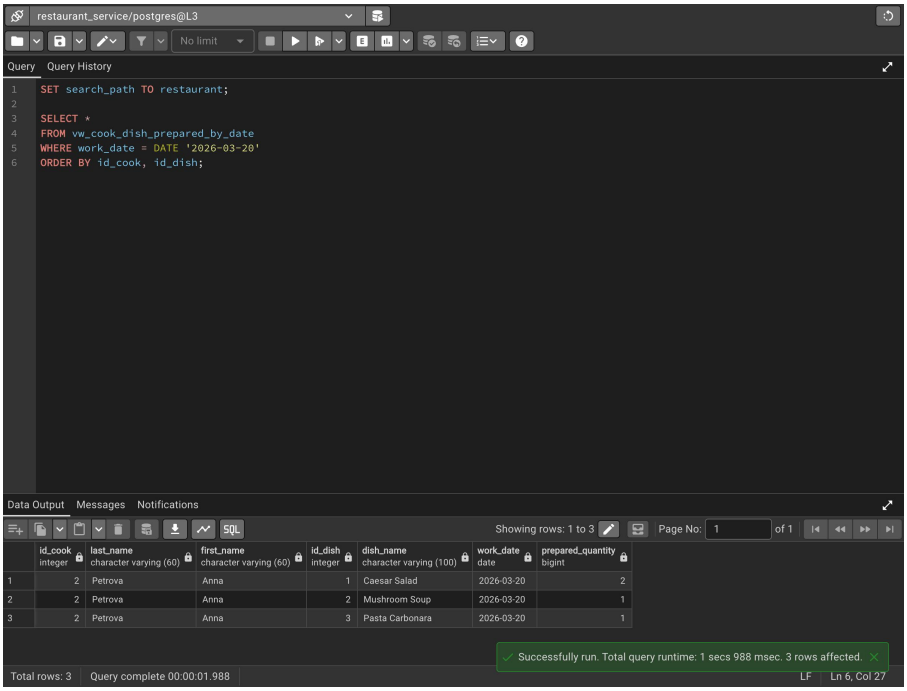
Листинг 12. Просмотр содержимого представления vw_cook_dish_prepared_by_date

```
SET search_path TO restaurant;

SELECT *
FROM vw_cook_dish_prepared_by_date
ORDER BY work_date, id_cook, id_dish;
```



Листинг 13. Просмотр данных представления за дату 2026-03-20

```
SET search_path TO restaurant;

SELECT *
FROM vw_cook_dish_prepared_by_date
WHERE work_date = DATE '2026-03-20'
ORDER BY id_cook, id_dish;
```



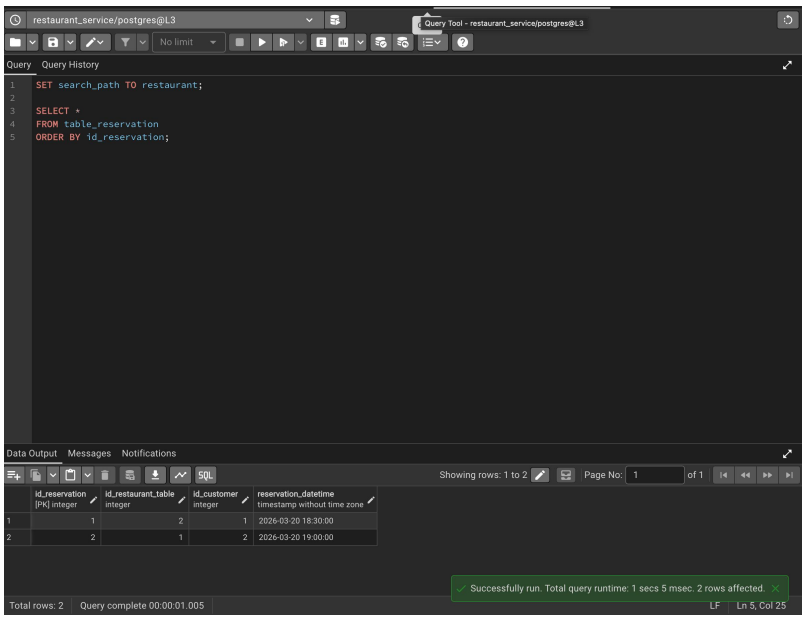
4.3. Запросы на модификацию данных

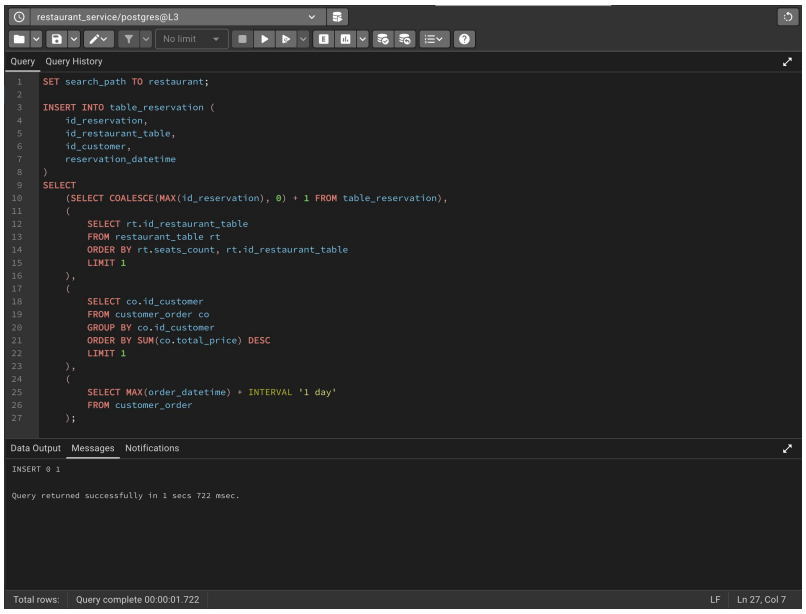
Для демонстрации запросов на модификацию данных использовались транзакции с командами BEGIN и ROLLBACK, что позволило получить скриншоты состояния данных до и после выполнения запросов без изменения итогового состояния БД.

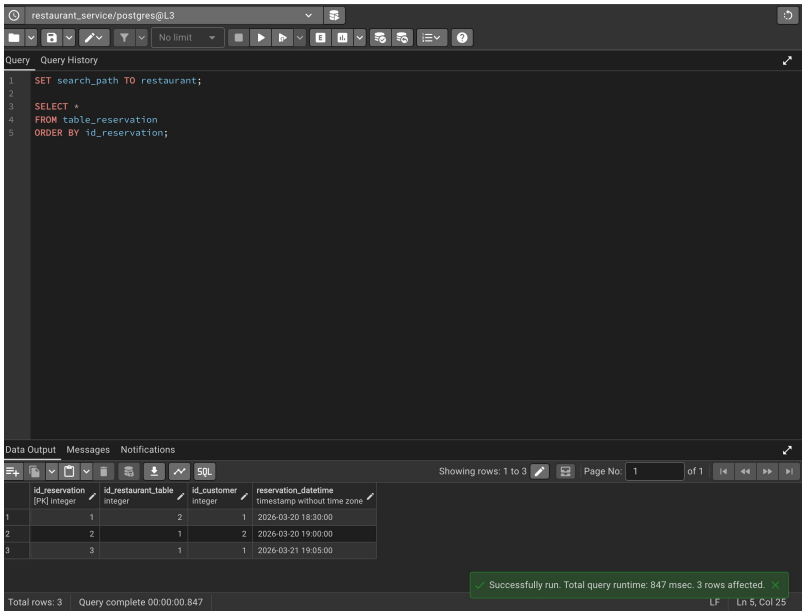
Запрос 1. Добавление новой брони для клиента с максимальной суммой заказов на стол с минимальным количеством мест

Листинг 14. Запрос INSERT с подзапросами для добавления новой брони

```
BEGIN;

SET search_path TO restaurant;

SELECT *
FROM table_reservation
ORDER BY id_reservation;

INSERT INTO table_reservation (
    id_reservation,
    id_restaurant_table,
    id_customer,
    reservation_datetime
)
SELECT
    (SELECT COALESCE(MAX(id_reservation), 0) + 1 FROM table_reservation),
    (
        SELECT rt.id_restaurant_table
        FROM restaurant_table rt
        ORDER BY rt.seats_count, rt.id_restaurant_table
        LIMIT 1
    ),
    (
        SELECT co.id_customer
        FROM customer_order co
        GROUP BY co.id_customer
        ORDER BY SUM(co.total_price) DESC
        LIMIT 1
    ),
    (
        SELECT MAX(order_datetime) + INTERVAL '1 day'
        FROM customer_order
    );

SELECT *
FROM table_reservation
ORDER BY id_reservation;

ROLLBACK;
```


restaurant_service/postgres@L3
Query Tool - restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
SELECT *
FROM table_reservation
ORDER BY id_reservation;
Data Output
Messages
Notifications
Showing rows: 1 to 2
Page No: 1 of 1
id_reservation [PK] integer
id_restaurant_table integer
id_customer integer
reservation_datetime timestamp without time zone
1 1 2 1 2026-03-20 18:30:00
2 2 1 2 2026-03-20 19:00:00
Successfully run. Total query runtime: 1 secs 5 msec. 2 rows affected.
Total rows: 2
Query complete 00:00:01.005
LF
Ln 5, Col 25

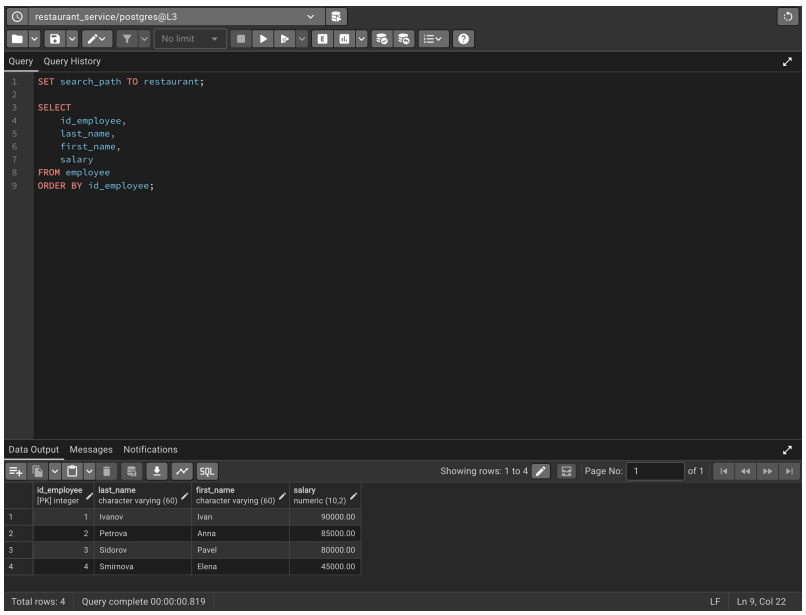

restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
INSERT INTO table_reservation (
id_reservation,
id_restaurant_table,
id_customer,
reservation_datetime
)
SELECT
(SELECT COALESCE(MAX(id_reservation), 0) + 1 FROM table_reservation),
(
SELECT rt.id_restaurant_table
FROM restaurant_table rt
ORDER BY rt.seats_count, rt.id_restaurant_table
LIMIT 1
),
(
SELECT co.id_customer
FROM customer_order co
GROUP BY co.id_customer
ORDER BY SUM(co.total_price) DESC
LIMIT 1
),
(
SELECT MAX(order_datetime) + INTERVAL '1 day'
FROM customer_order
);
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 1 secs 722 msec.
Total rows:
Query complete 00:00:01.722
LF
Ln 27, Col 7

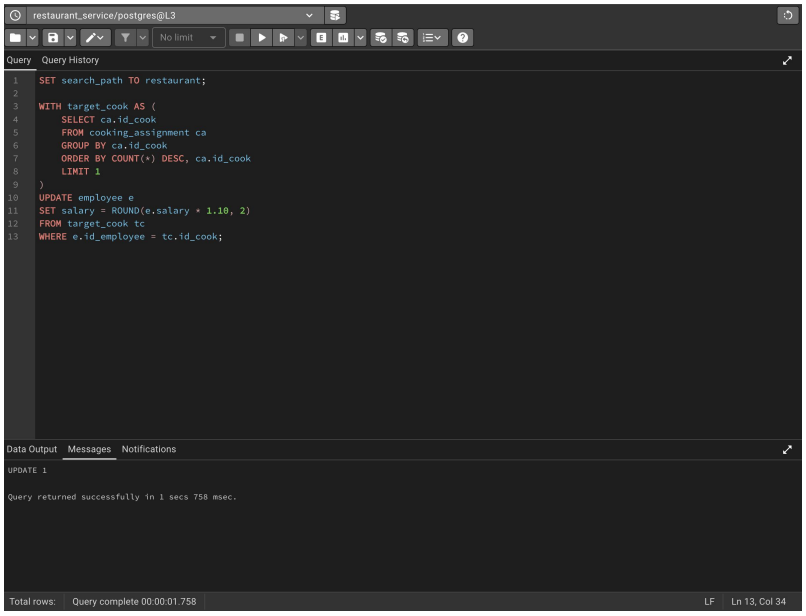

restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
SELECT *
FROM table_reservation
ORDER BY id_reservation;
Data Output
Messages
Notifications
Showing rows: 1 to 3
Page No: 1 of 1
id_reservation [PK] integer
id_restaurant_table integer
id_customer integer
reservation_datetime timestamp without time zone
1 1 2 1 2026-03-20 18:30:00
2 2 1 2 2026-03-20 19:00:00
3 3 1 1 2026-03-21 19:05:00
Successfully run. Total query runtime: 847 msec. 3 rows affected.
Total rows: 3
Query complete 00:00:00.847
LF
Ln 5, Col 25

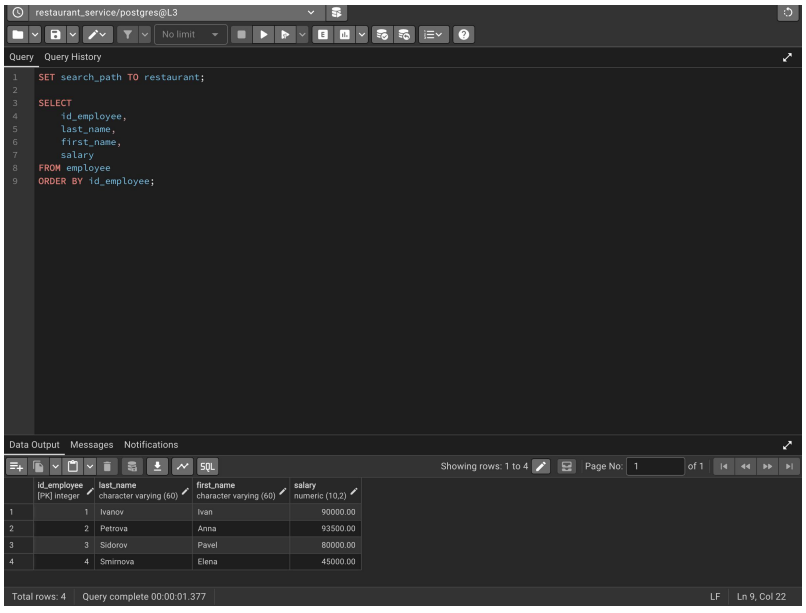
Запрос 2. Повышение оклада повару с максимальным числом назначений на приготовление

Листинг 15. Запрос UPDATE с подзапросом для изменения оклада сотрудника

```
BEGIN;

SET search_path TO restaurant;

SELECT
    id_employee,
    last_name,
    first_name,
    salary
FROM employee
ORDER BY id_employee;

WITH target_cook AS (
    SELECT ca.id_cook
    FROM cooking_assignment ca
    GROUP BY ca.id_cook
    ORDER BY COUNT(*) DESC, ca.id_cook
    LIMIT 1
)
UPDATE employee e
SET salary = ROUND(e.salary * 1.10, 2)
FROM target_cook tc
WHERE e.id_employee = tc.id_cook;

SELECT
    id_employee,
    last_name,
    first_name,
    salary
FROM employee
ORDER BY id_employee;

ROLLBACK;
```


restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
SELECT
id_employee,
last_name,
first_name,
salary
FROM employee
ORDER BY id_employee;
Data Output
Messages
Notifications
Showing rows: 1 to 4
Page No: 1
of 1
id_employee [PK] integer
last_name character varying (60)
first_name character varying (60)
salary numeric (10,2)
1 1 Ivanov Ivan 90000.00
2 2 Petrova Anna 85000.00
3 3 Sidorov Pavel 80000.00
4 4 Smirnova Elena 45000.00
Total rows: 4
Query complete 00:00:00.819
LF
Ln 9, Col 22

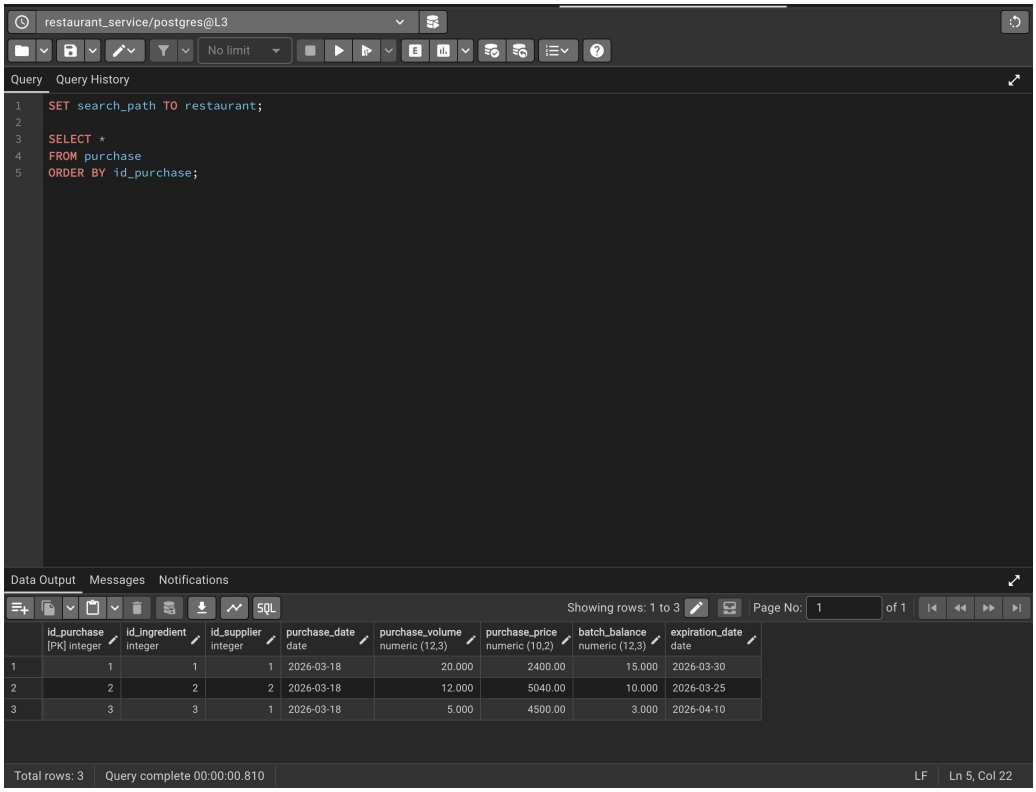

restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
WITH target_cook AS (
SELECT ca.id_cook
FROM cooking_assignment ca
GROUP BY ca.id_cook
ORDER BY COUNT(*) DESC, ca.id_cook
LIMIT 1
)
UPDATE employee e
SET salary = ROUND(e.salary * 1.10, 2)
FROM target_cook tc
WHERE e.id_employee = tc.id_cook;
Data Output
Messages
Notifications
UPDATE 1
Query returned successfully in 1 secs 758 msec.
Total rows:
Query complete 00:00:01.758
LF
Ln 13, Col 34

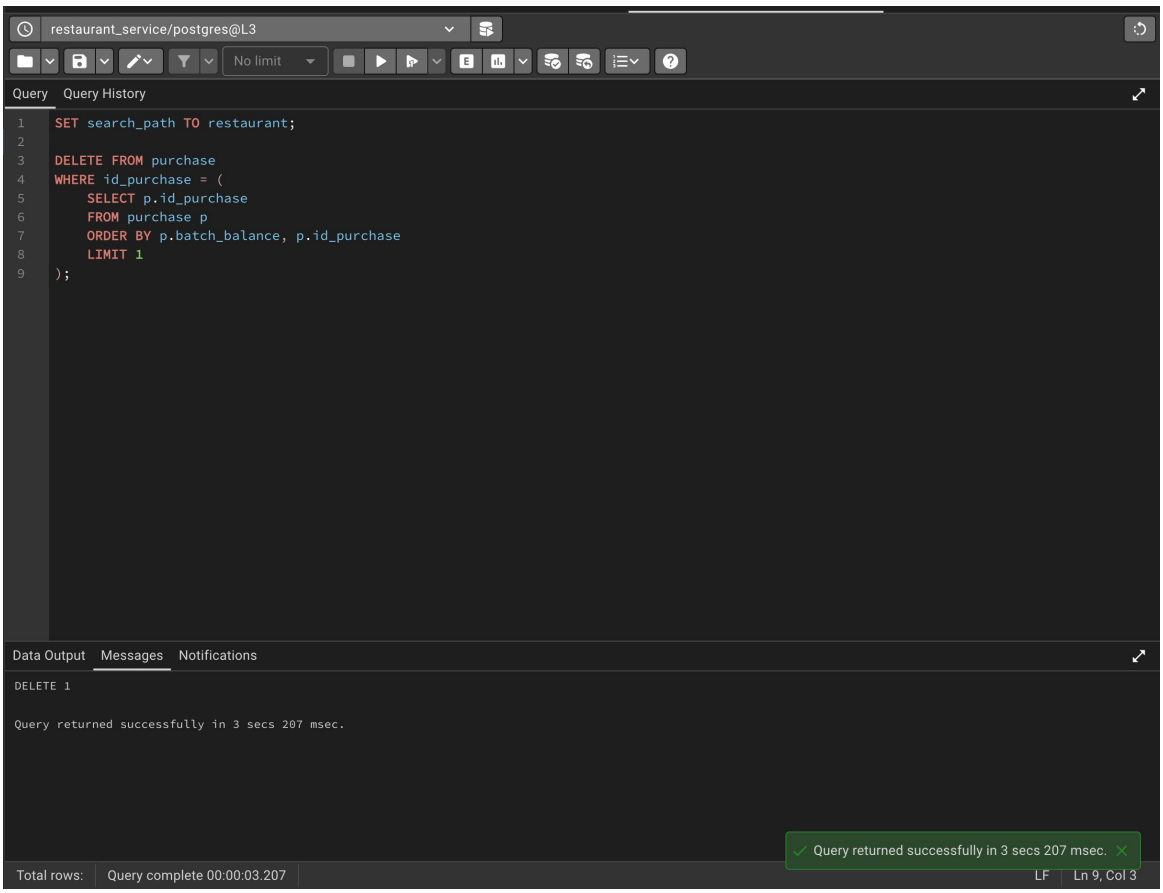

restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
SELECT
id_employee,
last_name,
first_name,
salary
FROM employee
ORDER BY id_employee;
Data Output
Messages
Notifications
Showing rows: 1 to 4
Page No: 1
of 1
id_employee [PK] integer
last_name character varying (60)
first_name character varying (60)
salary numeric (10,2)
1 1 Ivanov Ivan 90000.00
2 2 Petrova Anna 93500.00
3 3 Sidorov Pavel 80000.00
4 4 Smirnova Elena 45000.00
Total rows: 4
Query complete 00:00:01.377
LF
Ln 9, Col 22

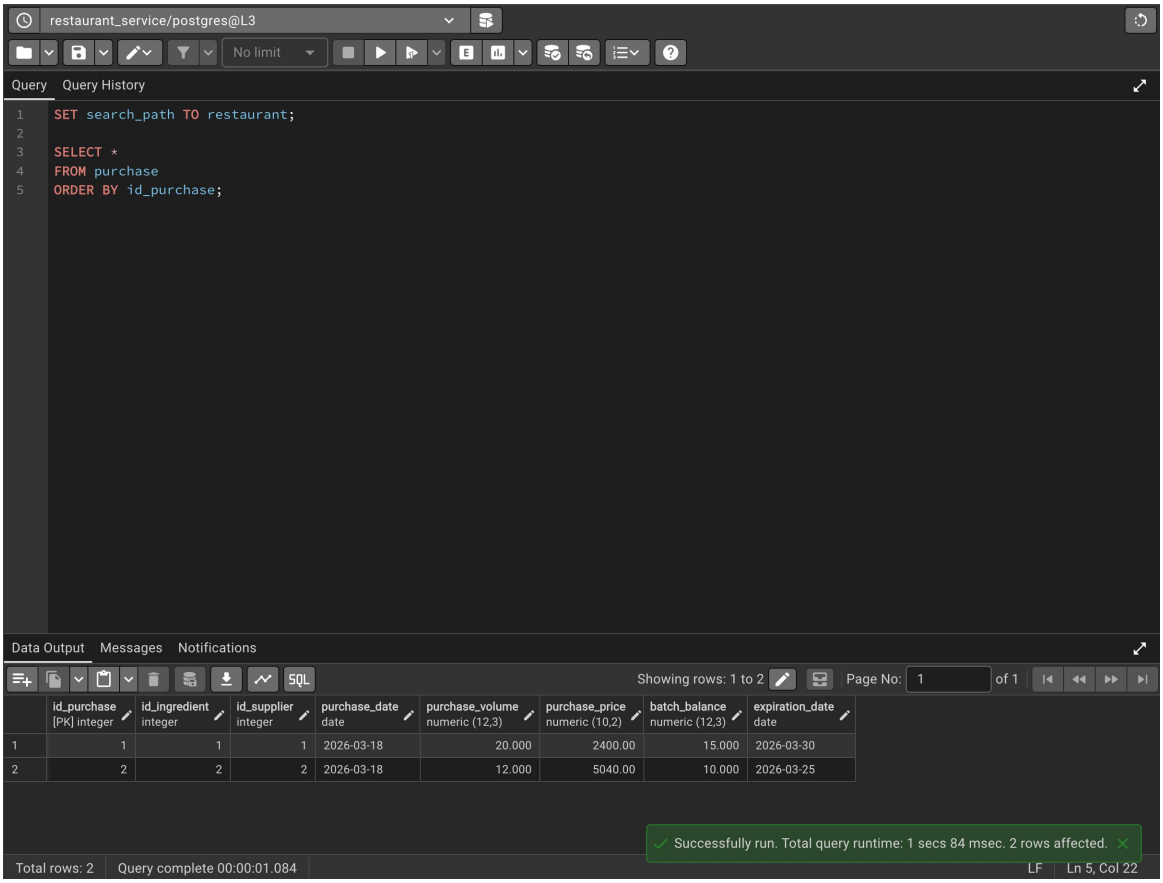
Запрос 3. Удаление закупки с минимальным остатком партии

Листинг 16. Запрос DELETE с подзапросом для удаления закупки с минимальным остатком

```
BEGIN;

SET search_path TO restaurant;

SELECT *
FROM purchase
ORDER BY id_purchase;

DELETE FROM purchase
WHERE id_purchase = (
    SELECT p.id_purchase
    FROM purchase p
    ORDER BY p.batch_balance, p.id_purchase
    LIMIT 1
);

SELECT *
FROM purchase
ORDER BY id_purchase;

ROLLBACK;
```

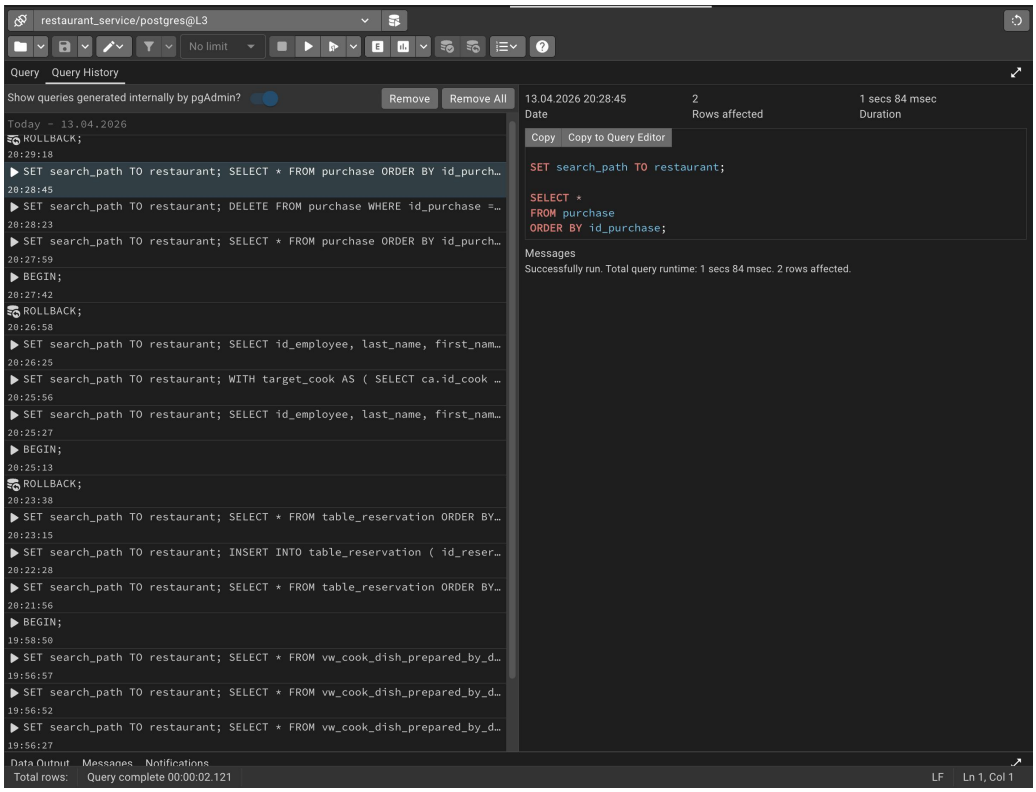


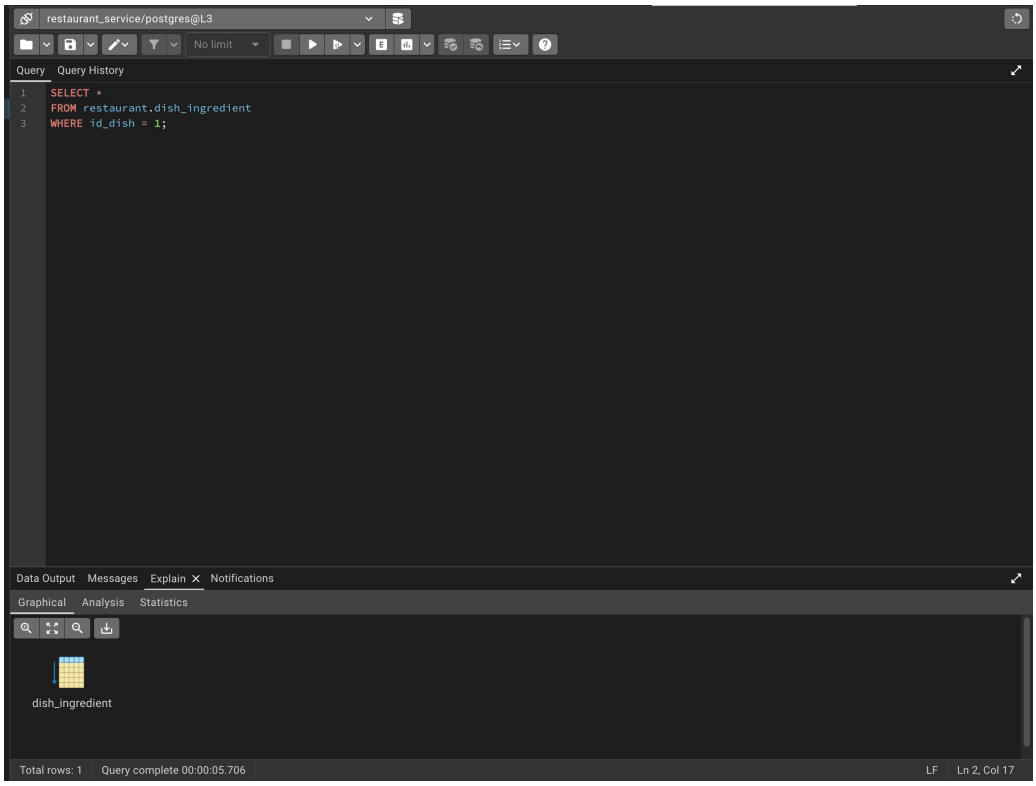

restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
DELETE FROM purchase
WHERE id_purchase = (
SELECT p.id_purchase
FROM purchase p
ORDER BY p.batch_balance, p.id_purchase
LIMIT 1
);
Data Output
Messages
Notifications
DELETE 1
Query returned successfully in 3 secs 207 msec.
Query returned successfully in 3 secs 207 msec.
Total rows:
Query complete 00:00:03.207
LF
Ln 9, Col 3


restaurant_service/postgres@L3
No limit
Query
Query History
SET search_path TO restaurant;
SELECT *
FROM purchase
ORDER BY id_purchase;
Data Output
Messages
Notifications
Showing rows: 1 to 2
Page No:
1
of 1
id_purchase [PK] integer
id_ingredient integer
id_supplier integer
purchase_date date
purchase_volume numeric (12,3)
purchase_price numeric (10,2)
batch_balance numeric (12,3)
expiration_date date
1 1 1 1 2026-03-18 20.000 2400.00 15.000 2026-03-30
2 2 2 2 2026-03-18 12.000 5040.00 10.000 2026-03-25
Successfully run. Total query runtime: 1 secs 84 msec. 2 rows affected.
Total rows: 2
Query complete 00:00:01.084
LF
Ln 5, Col 22

4.4. История запросов и графическое представление плана выполнения

В ходе работы был использован инструмент `Query History` для просмотра истории команд текущего сеанса, а также вкладка `Explain` для анализа графического представления плана выполнения запросов.





4.5. Создание индексов и сравнение времени выполнения запросов

В рамках лабораторной работы были исследованы два запроса:

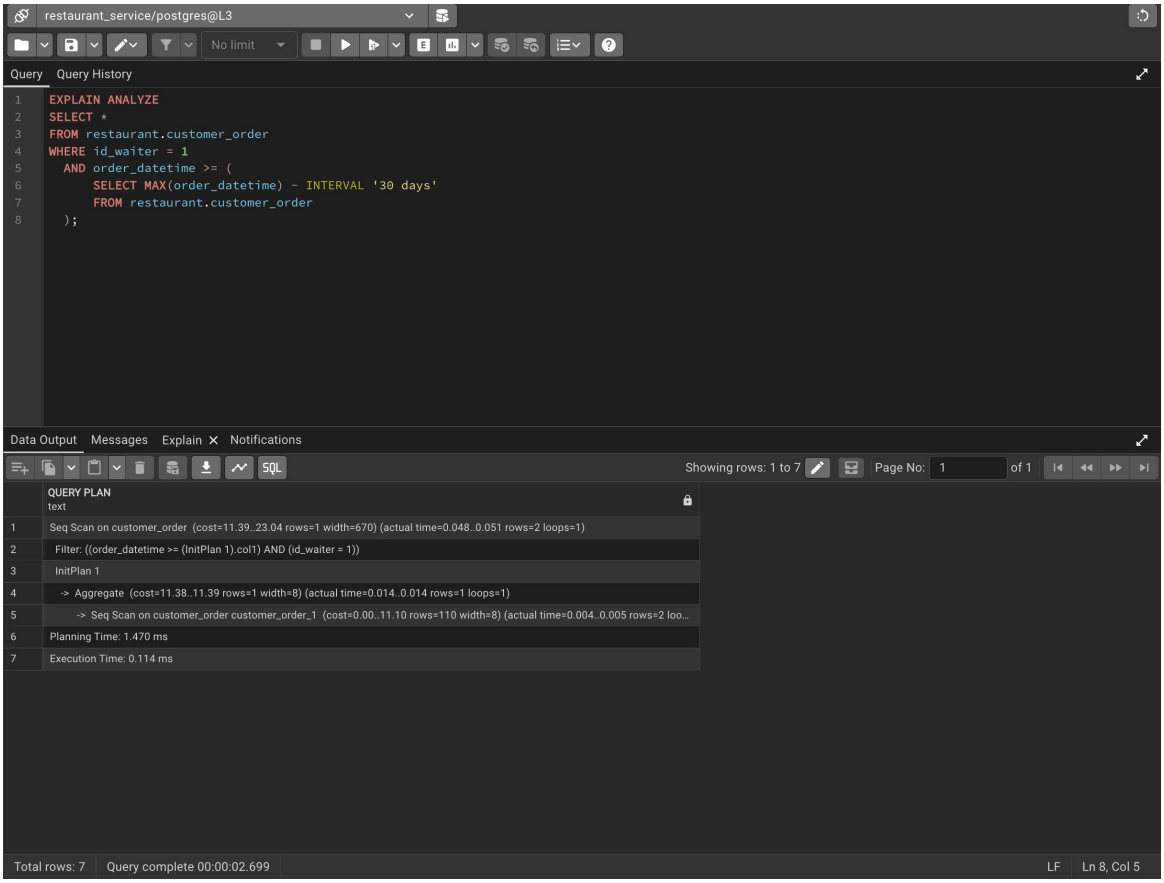
- запрос к таблице `customer_order` с фильтрацией по официанту и периоду времени;
- запрос к таблице `dish_ingredient` с фильтрацией по полю `id_dish`.

Для первого запроса был создан составной индекс, а для второго — простой индекс.

Составной индекс

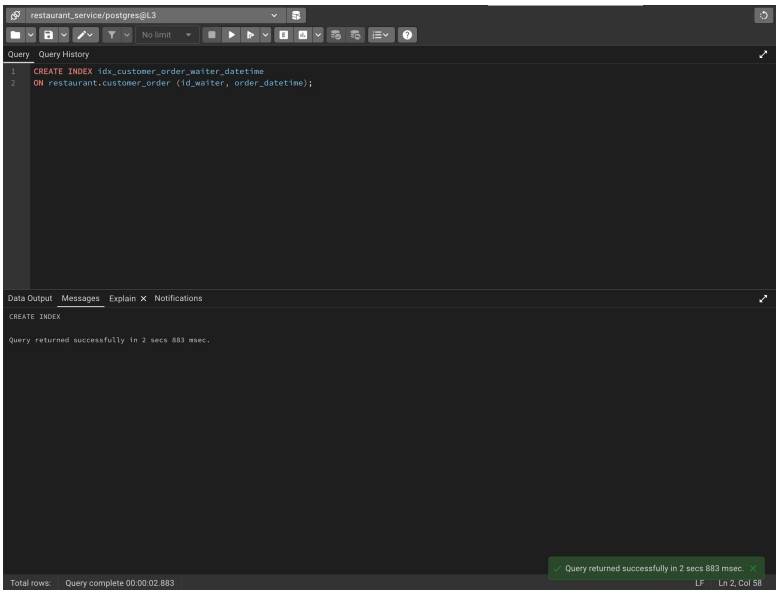
Листинг 17. Запрос к таблице customer_order без индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.customer_order
WHERE id_waiter = 1
  AND order_datetime >= (
      SELECT MAX(order_datetime) - INTERVAL '30 days'
      FROM restaurant.customer_order
  );
```



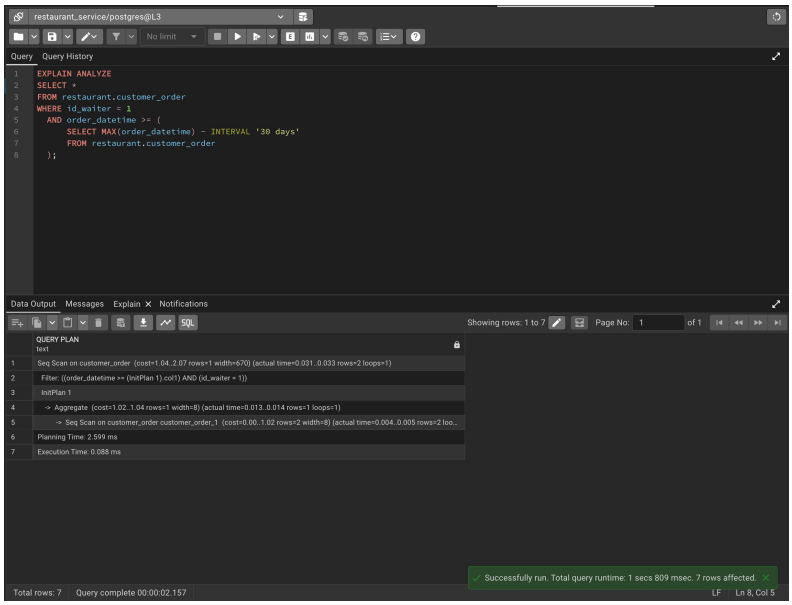
Листинг 18. Создание составного индекса для таблицы customer_order

```
CREATE INDEX idx_customer_order_waiter_datetime
ON restaurant.customer_order (id_waiter, order_datetime);
```



Листинг 19. Запрос к таблице customer_order после создания составного индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.customer_order
WHERE id_waiter = 1
  AND order_datetime >= (
      SELECT MAX(order_datetime) - INTERVAL '30 days'
      FROM restaurant.customer_order
  );
```



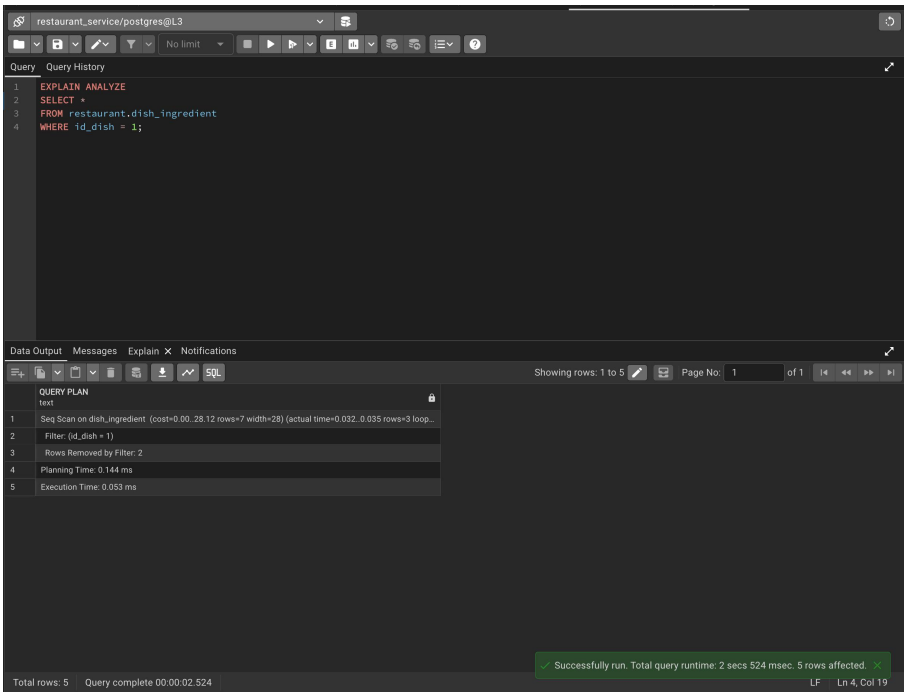
Листинг 20. Удаление составного индекса

```
DROP INDEX restaurant.idx_customer_order_waiter_datetime;
```

Простой индекс

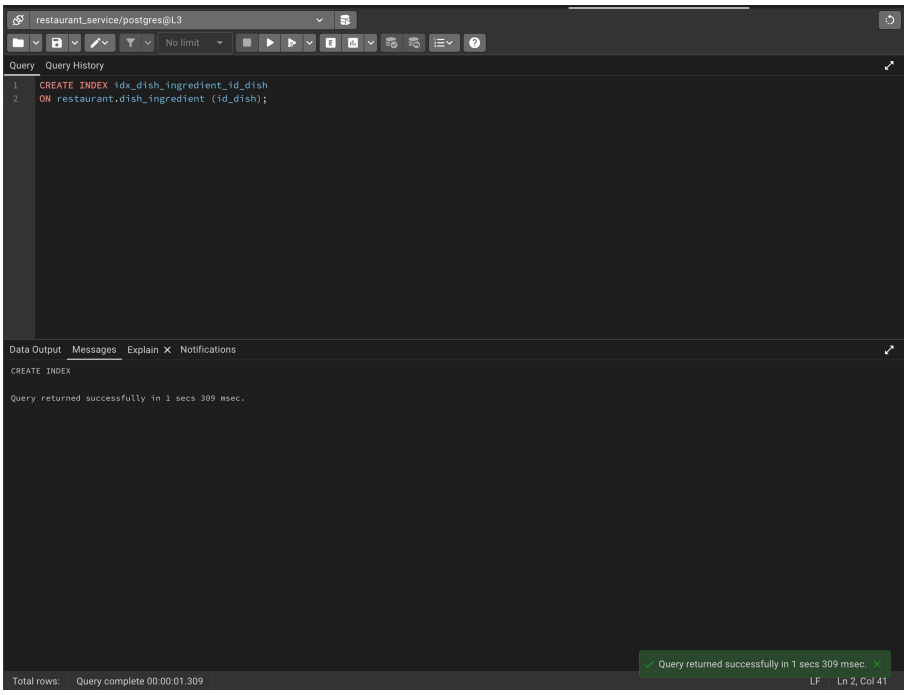
Листинг 21. Запрос к таблице dish_ingredient без индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.dish_ingredient
WHERE id_dish = 1;
```



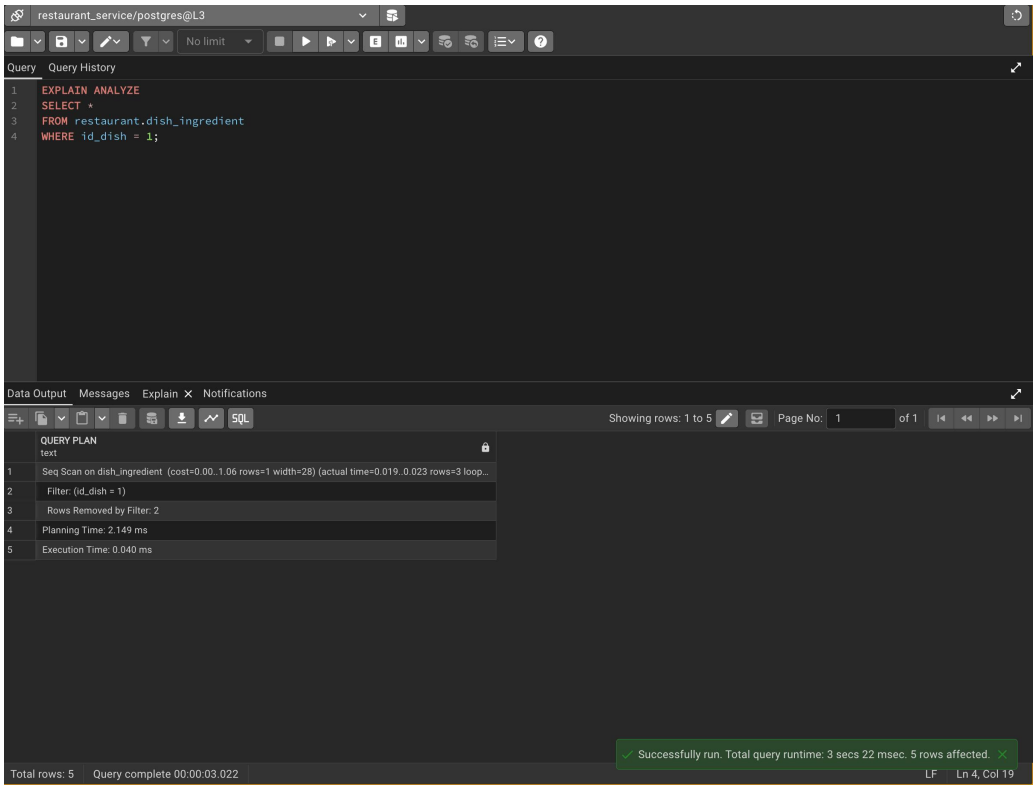
Листинг 22. Создание простого индекса для таблицы dish_ingredient

```
CREATE INDEX idx_dish_ingredient_id_dish
ON restaurant.dish_ingredient (id_dish);
```



Листинг 23. Запрос к таблице dish_ingredient после создания простого индекса

```
EXPLAIN ANALYZE
SELECT *
FROM restaurant.dish_ingredient
WHERE id_dish = 1;
```



Листинг 24. Удаление простого индекса

```
DROP INDEX restaurant.idx_dish_ingredient_id_dish;
```

Сравнение времени выполнения запросов

Результаты сравнения времени выполнения запросов представлены в таблице 1.

Таблица 1. Сравнение времени выполнения запросов без индекса и с индексом

Запрос	**Без индекса**	**С индексом**
Запрос к `customer_order`	0.114 ms	0.088 ms
Запрос к `dish_ingredient`	0.053 ms	0.040 ms

По результатам анализа видно, что в обоих случаях PostgreSQL продолжил использовать `Seq Scan`. Это объясняется тем, что объём тестовых данных в таблицах невелик, поэтому последовательное сканирование оказалось достаточно эффективным. При увеличении объёма данных использование индексов будет более значимым с точки зрения производительности.

5. Выводы

В ходе выполнения лабораторной работы были созданы запросы на выборку данных к базе данных PostgreSQL, построены представления, составлены запросы на модификацию данных с использованием подзапросов, изучены средства `Query History` и `EXPLAIN`, а также исследовано влияние индексов на время выполнения запросов.

В результате выполнения работы были приобретены практические навыки:

- составления запросов на выборку данных к базе данных PostgreSQL;
- создания и использования представлений;
- применения подзапросов в запросах на модификацию данных;
- анализа истории запросов и графического плана выполнения;
- создания простых и составных индексов;
- сравнительного анализа времени выполнения запросов без индексов и с индексами.

Таким образом, цель лабораторной работы достигнута: получены практические навыки работы с запросами на выборку и модификацию данных, представлениями и индексами в PostgreSQL.